GENIE
Security Architecture & Implementation Reference

Multi-Agent Financial Assistant
Built on Microsoft MARA · RBI FREE-AI Aligned

Version	Q1 2026 Hardening Release
Classification	Internal — Security Team + Regulators
Audience	CISO · Risk Officer · Security Architect · Platform Engineer
Repository	github.com/c2siorg/genie
Generated	May 28, 2026
Compliance	RBI FREE-AI 7 Sutras · 26 Recommendations · Annexure VI

This document is the canonical single reference for Genie's security implementation. Every claim is tied to a file path, function name, or test name in the repository. All eleven security layers are covered: from TLS termination and JWT verification through Postgres Row-Level Security, OAuth 2.0 Token Exchange, agent tier gating, governance policy evaluation, envelope encryption, tamper-evident audit, and BCP drills.

1. Executive Summary

Genie is an open-source, RBI FREE-AI aligned multi-agent financial assistant built on Microsoft's Multi-Agent Reference Architecture (MARA). It operates as a production-grade agentic system where multiple specialised AI agents collaborate to answer complex financial queries, orchestrate payments, run KYC checks, and generate regulatory reports — all while maintaining strict data isolation, tamper-evident audit trails, and formal governance over which agents are allowed to serve live customer traffic.

The Q1 2026 hardening release shipped four security primitives that close the gaps identified against RBI FREE-AI Recommendations 15, 17, 22, and 23:

Primitive	What it does	File(s)
Postgres Row-Level Security (RLS)	Database-enforced tenant isolation. Even if application code has a bug, Postgres refuses to return another tenant's rows.	pkg/storage/postgres/migrations/0005_rls.sql pkg/storage/postgres/tenant.go
OAuth 2.0 Token Exchange (RFC 8693)	Dual-identity JWT tokens. The Subject stays the user; the Actor records the agent acting on their behalf. Full audit chain across N-hop agent calls.	pkg/auth/tokenexchange/exchange.go pkg/auth/types.go · pkg/auth/jwt.go
Agent Tier Promotion Model	Four-stage gate: Sketch → Prototype → Beta → Production. Only Production agents serve live customer traffic. Defaults fail-closed to Prototype.	pkg/agent/tier.go
Governance TenantPolicy	Bus-layer message filtering. Every message crossing the in-memory bus must carry a matching tenant_id / expected_tenant pair or be denied.	pkg/governance/tenant.go

Security posture: defence in depth across 11 layers. 104 Go packages pass `go test ./...`. 110 UI contract tests. 8 integration tests wire all four primitives together end-to-end.

2. Defence in Depth — The 11-Layer Security Envelope

Single-line defences fail. A WHERE user_id = $1 clause that someone forgot to add is a cross-tenant leak. An if !isAdmin { return 403 } check that someone copied wrong is privilege escalation. Genie's security envelope assumes every layer has bugs — the job of the stack is to ensure a bug in any one layer is contained by the next.

The following 11 layers execute in order for every /v1/ask request:

#	Layer	File(s)	What it stops
L1	TLS Termination	Provided by ingress (out of repo scope)	Network eavesdropping, MITM
L2	HTTP Middleware	pkg/web/mid/ratelimit.go pkg/web/mid/tracing.go	DoS, request floods, missing trace context
L3	JWT Verify + Claims Extract	pkg/web/mid/auth.go → pkg/auth/jwt.go::Verify()	Unauthenticated callers, forged tokens, expired sessions
L4	Role Gate	pkg/web/mid/auth.go::RequireRole()	Privilege escalation to admin-only endpoints
L5	Handler Validation	pkg/web/handlers/*.go	Schema violations, oversized payloads, bad classifications
L6	Message Enrichment	pkg/web/handlers/ask.go	Missing tenant_id / user_roles metadata on bus messages
L7	CompositePolicy on Bus	pkg/orchestration/orchestrator.go pkg/governance/*.go	RBAC, cross-tenant, below-tier, injection, PII, schema, consent
L8	Agent HandleMessage	agents//.go	Domain-specific validation; deterministic core logic
L9	DB Query under WithTenant	pkg/storage/postgres/tenant.go migrations/0005_rls.sql	Cross-tenant DB reads even if L7 had a bug
L10	LLM Call Wrappers	pkg/llm/{deadline,circuit,budget,router}.go	Runaway cost, latency, sovereignty violations
L11	Audit + Hash Chain + Incident	pkg/compliance/audit.go pkg/incidents/incidents.go	Tamper-evident record; Annexure VI auto-filing

The key reviewer contract: no two adjacent layers share a single point of failure for the same attack class. A cross-tenant request bypassing L7 still hits L9. An unauthorised caller bypassing L4 still hits L3. A hallucinating LLM bypassing L7's classification check still gets the disclaimer added at L11.

3. Threat Model

The following 15 threats are actively designed for. STRIDE categories anchor each threat to standard literature. Every threat maps to one or more mitigating controls in the codebase.

ID	Threat	STRIDE	Mitigation	Risk
T1	Cross-tenant data leak via missing WHERE clause	Info Disclosure	RLS (L9) + bus TenantPolicy (L7)	HIGH
T2	Privilege escalation via forged / replayed JWT	Spoofing	Short TTL + HS256 signature + audience check; passkeys for MFA	HIGH
T3	Confused-deputy: agent A's token reads agent B's data	Elevation of Priv	RFC 8693 audience-scoped exchanged tokens	HIGH
T4	Prompt injection pivoting a tool call	Tampering	PromptInjectionPolicy + output schema validation + tool allowlist	HIGH
T5	Hallucinated payment / KYC verdict	Tampering	Deterministic agent core; LLM is narration-only layer	HIGH
T6	Audit log tampering by insider with DB write access	Repudiation	Hash chain in pkg/compliance/audit.go + WORM external sink	MEDIUM
T7	Untriaged AI-generated agent in prod traffic	Tampering / EoP	Tier promotion gate (L7) defaults to TierPrototype	HIGH
T8	LLM provider data-residency violation	Compliance	pkg/sovereignty.ProviderRegistry.Allowed()	MEDIUM
T9	Runaway autonomy — agent loops till budget exhausted	Denial of Svc	Budget + circuit + deadline wrappers (L10)	MEDIUM
T10	Single-vendor outage takes down customer path	Availability	Fallback agents + BCP drill	MEDIUM

T11	KEK compromise exposes all encrypted rows	Info Disclosure	Per-row kek_id, KMS-pluggable resolver, rotation playbook	HIGH
T12	Service-to-service identity spoofing inside mesh	Spoofing	SPIFFE/SVID identity (pkg/identity) + mTLS (infra layer)	MEDIUM
T13	Insider browsing all incidents via admin UI	Info Disclosure	Admin-only routes + audit on every read	MEDIUM
T14	Supply-chain attack on a third-party model provider	Tampering	AIBOM with provenance + pkg/safety adversarial corpus on release	MEDIUM
T15	Adversarial input evading safety scoring	Tampering	Plugin chain with all-of mode; multi-vendor scoring	MEDIUM

4. Identity Layer

Identity in Genie comes in three flavours, each with a different trust root and lifecycle.

4.1 User Identity (Humans)

Field	Type	Notes	Location
User.ID	UUID	Generated at signup	pkg/storage/postgres/users.go
User.Email	String (unique, lower-cased)	The login key; unique index enforced	users table
User.Roles	[]string	Multi-role array: user / advisor / admin	pkg/auth/types.go
User.Password Hash	bcrypt	Raw password never stored; cost factor 12	pkg/auth/jwt.go

User records live in the users table in Postgres, itself protected by RLS. The RLS policy includes an __admin__ OR clause so the login handler (which runs without a user session) can look up by email without bypassing the policy.

4.2 Agent Identity (Software within Genie)

Every agent declares a stable string ID at the top of its package:

```
// agents/kyc_orchestrator/kyc_orchestrator.go
const (
ID = "kyc_orchestrator"
Name = "KYC Orchestrator"
)
```

The registry indexes by ID. tests/agents_registry/ enforces uniqueness across the entire agent tree via a grep test that runs on every go test ./... . The agent ID is what Actor.Subject carries in the RFC 8693 dual-identity flow.

4.3 Workload Identity (Services Across the Network)

pkg/identity ships SPIFFE-style DID + W3C Verifiable Credential primitives using Ed25519 keys:

```
// pkg/identity/identity.go
func NewDIDKey() (*DID, error) // did:key:z... from Ed25519 pair
func IssueVC(issuer *DID, vc *VerifiableCredential) (*VerifiableCredential, error)
func VerifyVC(vc *VerifiableCredential, pub ed25519.PublicKey) error
```

DIDs let a service prove 'I am kyc-mcp-server' without needing a shared secret. The credential model lets the issuer attest scoped claims. Full SPIFFE/SPIRE wiring at deploy time is the roadmap item connecting this to mTLS.

5. Authentication

5.1 JWT (HS256) — Primary Session Token

Genie implements its own minimal JWT rather than pulling in a third-party library. The rationale: the surface area needed is small (HS256, exp, iat, aud), and third-party JWT libraries have a documented history of alg=none and RSA key-confusion vulnerabilities. The entire HS256 path is ~150 lines of stdlib: crypto/hmac, crypto/sha256, encoding/base64, encoding/json.

File: pkg/auth/jwt.go

```
type Issuer struct {
secret []byte
issuer string
audience []string
ttl time.Duration
}

// Issue mints a first-party user token.
func (i *Issuer) Issue(userID, email string, roles []Role) (string, *Claims, error)

// IssueWithActor mints an RFC 8693 dual-identity token.
func (i *Issuer) IssueWithActor(userID, email string, roles []Role,
audience []string, actor *Actor) (string, *Claims, error)

// Verify checks signature, exp, and audience.
func (i *Issuer) Verify(token string) (*Claims, error)

// VerifyIgnoringAudience is used by token-exchange for multi-hop chains.
func (i *Issuer) VerifyIgnoringAudience(token string) (*Claims, error)
```

Claims Structure

```
type Claims struct {
Subject string `json:"sub"`
Email string `json:"email"`
Roles []Role `json:"roles"`
Issuer string `json:"iss"`
Audience []string `json:"aud"`
IssuedAt int64 `json:"iat"`
Expires int64 `json:"exp"`
// Actor is populated by token-exchange (RFC 8693).
Actor *Actor `json:"act,omitempty"`
}

type Actor struct {
Subject string `json:"sub"`
Issuer string `json:"iss,omitempty"`
Nested *Actor `json:"act,omitempty"` // N-hop chain
}
```

5.2 OAuth Device Flow (RFC 8628)

For headless / CLI deployments (e.g. a data-pipeline script calling the Genie API), the system supports OAuth 2.0 Device Authorization Grant. File: pkg/auth/device_flow.go. The flow issues a device_code, polls for user approval, then exchanges for a short-lived access token. Tokens issued via device flow carry the device_flow scope so downstream handlers can apply additional restrictions.

5.3 WebAuthn / Passkeys (Ed25519)

For the admin console and elevated-privilege operations, Genie supports WebAuthn passkeys using Ed25519 authenticators. File: pkg/auth/webauthn.go. Passkeys eliminate the password-spray attack surface on admin accounts. Challenge-response is implemented using crypto/rand challenges; the server stores the credential public key, not the private key.

5.4 Privileged Access Manager (PAM)

pkg/auth/elevation implements a two-person-rule privileged access flow analogous to Google Cloud's Privileged Access Manager. Key properties:

- A requester submits a grant request with subject, reason, role, and TTL.
- A minimum number of approvers (N-eyes, default 2) must approve before the grant activates.
- Self-approval is rejected at the code level.
- Only admin-role users may approve.
- Every request and approval writes a line to the audit log.

```
// pkg/auth/elevation/elevation.go
func (s *Service) Request(ctx context.Context, req ElevationRequest) (GrantID, error)
func (s *Service) Approve(ctx context.Context, grantID GrantID, approverID string) error
func (s *Service) ActiveGrants(subject string) []Grant
```

6. Authorisation — RBAC, Tier, and the Policy Stack

6.1 Role-Based Access Control

Three roles are defined in pkg/auth/types.go:

Role	Capabilities
user	End customer. Can ask questions, upload documents, view own accounts.
advisor	Relationship manager. Can view customer records they are assigned to.
admin	Platform operator. Can access /v1/ai-inventory, /v1/aibom, /v1/incidents; can promote agents, run BCP drills, view cross-tenant audit logs.

RBACPolicy (pkg/governance/rbac.go) is the bus-layer enforcement of these roles. It checks the user_roles metadata field on every message and denies messages whose type requires a role the caller doesn't hold. AdminBypass is an explicit opt-in per policy instance — not a default.

6.2 Agent Tier Promotion Model

Every agent in Genie advertises a Tier via the TierAware interface. File: pkg/agent/tier.go.

```
type Tier string

const (
TierSketch Tier = "sketch" // AI-generated; sandbox only
TierPrototype Tier = "prototype" // engineer-owned; no production
TierBeta Tier = "beta" // limited production; monitored
TierProduction Tier = "production" // fully promoted; customer traffic OK
)

type TierAware interface {
Tier() Tier
}

// TierOf returns TierPrototype for agents that don't declare a tier.
// Default fail-closed: undeclared = not production.
func TierOf(a agent.Agent) Tier

// Production returns true only for TierProduction.
func Production(t Tier) bool

// AtLeast returns true if got >= required in the promotion ordering.
func AtLeast(got, required Tier) bool
```

Tier	Meaning and Constraints

Sketch	AI-generated prototype. Sandbox / demo only. Never dispatched to live customer traffic. Useful for quick exploration and prototyping.
Prototype	Engineer-owned, manually reviewed, but not yet security-reviewed or red-teamed. Default for any agent that doesn't declare a tier. Cannot serve customer-facing traffic.
Beta	Security-reviewed, has fallback, has audit hooks. Limited production with monitoring and staged rollout. Requires risk-team sign-off.
Production	Fully promoted. Red-teamed, fallback verified, BCP-drilled. The only tier allowed to serve live customer traffic by default.

The TierPolicy (local to tests/security_envelope_test.go) demonstrates how the tier check integrates into the CompositePolicy. In production, the tier gate is enforced at agent registration time (cmd/api) so mismatched agents never appear in the registry.

6.3 CompositePolicy — The Full Policy Stack

The orchestrator evaluates every bus message through a CompositePolicy that chains all governance checks. File: pkg/governance/compliance.go.

#	Policy	What it checks
1	RBACPolicy	Does the caller hold the required role for this message type?
2	TenantPolicy	Does tenant_id == expected_tenant? Is tenant_id present?
3	TierPolicy	Is the target agent at TierProduction?
4	ClassificationPolicy	Is the document classification appropriate for this agent?
5	PromptInjectionPolicy	Does the message contain injection patterns?
6	PIIPolicy	Does the message contain unredacted PII that shouldn't cross this boundary?
7	SchemaPolicy	Does the message conform to the declared schema for its type?
8	ConsentPolicy	Has the user given consent for the data processing this message triggers?
9	ExplainabilityPolicy	Does the message type require an explanation token?
10	MaxContentLengthPolicy	Is the message within the configured size limit?
11	BoardDSL rules	All rules loaded from pkg/policy/dsl/*.yaml (board-approved policies)

7. Tenant Isolation — Bus Layer + Database Layer

Defence in depth: a bug in the bus check is caught by RLS at the DB; a misconfiguration in RLS is caught by the bus check. Neither layer alone is sufficient — both together provide the guarantee.

7.1 Bus-Layer: TenantPolicy

File: pkg/governance/tenant.go. The TenantPolicy is evaluated on every message that crosses the in-memory message bus before it reaches any agent.

```
type TenantPolicy struct {
// AppliesTo restricts the policy to specific message types.
// Leave nil to apply to every message.
AppliesTo []string
// AdminBypass allows messages with user_roles.admin to skip the check.
// Default false — must be explicitly opted in.
AdminBypass bool
}

func (p TenantPolicy) Evaluate(ctx context.Context, msg protocol.Message)
(governance.PolicyResult, error)
```

Denial conditions:

- metadata.tenant_id is missing or empty string.
- metadata.expected_tenant is present and does not equal tenant_id.
- The message type is in AppliesTo (or AppliesTo is nil — all types).

The policy is small by design. Cross-tenant routing is a boolean question — does the metadata match? A complex policy has more places for bugs.

7.2 Database Layer: Postgres RLS

Files: pkg/storage/postgres/tenant.go + migrations/0005_rls.sql. RLS moves the tenant filter into Postgres itself. No matter what SQL the application sends, Postgres refuses to return rows whose tenant column doesn't match the session's app.current_tenant GUC.

Migration — 0005_rls.sql

The migration enables FORCE ROW LEVEL SECURITY on every tenant table:

```
-- Enable and FORCE on every tenant-carrying table
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE documents FORCE ROW LEVEL SECURITY;

-- Standard tenant policy
CREATE POLICY documents_tenant_isolation ON documents
USING (user_id::text = current_setting('app.current_tenant', true)
OR current_setting('app.current_tenant', true) = '__admin__');

-- Same pattern for: accounts, mcp_tokens, incidents, users
```

FORCE extends the policy to table owners — without it, the migration role would silently bypass RLS. The CI pipeline verifies relforcerowsecurity=true after every migration run.

Go Layer — WithTenant

```
// pkg/storage/postgres/tenant.go

const AdminTenant = "__admin__"

var ErrNoTenant = errors.New("postgres: tenant id is required")

// WithTenant runs fn inside a txn with SET LOCAL for app.current_tenant.
func (db *DB) WithTenant(ctx context.Context, tenantID string,
fn TenantFunc) error {
if tenantID == "" {
return ErrNoTenant
}
return db.pool.BeginTxFunc(ctx, pgx.TxOptions{}, func(tx pgx.Tx) error {
_, err := tx.Exec(ctx,
"SELECT set_config('app.current_tenant', $1, true)", tenantID)
if err != nil { return err }
return fn(ctx, tx)
})
}

// WithAdminContext uses the __admin__ sentinel for cross-tenant reads.
// Use only at login and admin-only endpoints.
func (db *DB) WithAdminContext(ctx context.Context, fn TenantFunc) error {
return db.WithTenant(ctx, AdminTenant, fn)
}
```

Why SET LOCAL and not SET SESSION

SET LOCAL scopes the GUC to the current transaction only. It is automatically reverted when the transaction commits or rolls back, even if the application crashes mid-transaction. SET SESSION would persist across connections in a connection pool, creating a subtle cross-tenant leak when a connection is returned to the pool and reused by a different tenant's request.

8. OAuth 2.0 Token Exchange — RFC 8693 Dual-Identity Tokens

8.1 The Problem: Audit Gap in Agent Chains

When an agent calls an MCP server, the downstream service has historically faced two bad options:

- See only the user — receive the user's first-party token. Audit says 'user did X.' Loses the fact that an automated agent was the actual originator.
- See only the agent — receive a service-to-service token. Audit says 'kyc_orchestrator called X.' Loses the user — every audit row collapses onto the same service identity.

RFC 8693 provides the third option: dual-identity tokens. The Subject claim stays the user; an Actor (act) claim records the agent acting on the user's behalf.

8.2 Implementation

File: pkg/auth/tokenexchange/exchange.go

```
type Service struct {
verifier looseVerifier
minter *auth.Issuer
serviceIdentity string
safetyMargin time.Duration
mu sync.RWMutex
cache map[cacheKey]cacheEntry
}

type Request struct {
SubjectToken string // the user's existing JWT
ActorID string // the agent performing the action
Audience string // the downstream service being called
}

// Exchange mints a dual-identity token.
// Returns: (tokenString, *Claims, error)
func (s *Service) Exchange(ctx context.Context, req Request)
(string, *auth.Claims, error)

// Invalidate clears all cached tokens for a given user subject.
// Call on logout or password change.
func (s *Service) Invalidate(userSubject string)
```

8.3 Token Structure After Exchange

```
{
"sub": "user-alice", // UNCHANGED — always the human user
"email": "alice@example.com",
"roles": ["user"],
"aud": ["mcp://kyc-server"], // audience = downstream service
```

```
"act": { // RFC 8693 actor claim
"sub": "kyc_orchestrator" // agent acting on behalf of user
}
}
```

8.4 N-Hop Nested Actor Chains

For multi-hop flows (user → agent → MCP server → upstream API), Actor.Nested carries the full chain:

```
// Hop 1: user → kyc_orchestrator → mcp://kyc-server
hop1, _, _ := svc.Exchange(ctx, tokenexchange.Request{
SubjectToken: userToken,
ActorID: "kyc_orchestrator",
Audience: "mcp://kyc-server",
})

// Hop 2: same user → kyc-mcp-server → https://api.upstream/records
_, claims, _ := svc.Exchange(ctx, tokenexchange.Request{
SubjectToken: hop1,
ActorID: "kyc-mcp-server",
Audience: "https://api.upstream/records",
})

// claims.Subject == 'user-alice' (never changes)
// claims.Actor.Subject == 'kyc-mcp-server' (outermost)
// claims.Actor.Nested.Subject == 'kyc_orchestrator' (inner)
```

8.5 Caching and Invalidation

Exchanged tokens are cached by (user_subject, actor_id, audience) tuple. TTL = min(subject_token.exp, actor_token.exp) - safetyMargin. The Invalidate(userSubject) method clears all cache entries for a user on logout or password change. CacheSize() is exposed for testing.

8.6 LooseVerifier for Multi-Hop Chains

The second hop receives a token whose audience is the first hop's downstream service (e.g. mcp://kyc-server), not the issuer's default audience. A strict audience check would reject it. The looseVerifier wraps the Issuer and temporarily nils the Audience list during verification, allowing the exchange service to validate the signature without failing on audience mismatch.

9. Governance Policies as Code

All governance rules are expressed as Go structs implementing the Policy interface. There are no strings to inject, no config files to parse at runtime that could be tampered with. Every policy is compiled into the binary and version-controlled alongside the code it governs.

```
// pkg/governance/policy.go
type Policy interface {
Evaluate(ctx context.Context, msg protocol.Message)
(PolicyResult, error)
}

type PolicyResult struct {
Decision Decision // DecisionAllow | DecisionDeny
Reason string
CheckedAt time.Time
CheckedByID string
}
```

9.1 Board-Approved Policy DSL

pkg/policy/dsl implements a CEL-style policy language (stdlib only, no external dependencies) for loading board-approved rule files. A rule file defines allow/deny expressions over message metadata. These map to RBI FREE-AI Recommendation 14 (board-approved policy).

```
# board_policy.yaml — example rule
rules:
- id: block_cross_border_payments
type: finance_payment
deny_if: 'metadata.destination_country != "IN"'
reason: RBI cross-border restriction

- id: require_edd_for_pep
type: kyc_result
deny_if: 'metadata.pep_score > 0.5 && metadata.edd_completed == false'
reason: EDD mandatory for Politically Exposed Persons
```

9.2 Prompt Injection Policy

pkg/governance/prompt_injection.go evaluates every user-facing message for known injection patterns: role-override prefixes, base64-encoded instructions, XML-wrapped directives, and Unicode homoglyph substitutions. The scoring is deterministic — no LLM call in the hot path.

9.3 PII Policy

pkg/governance/pii.go detects PII classes (Aadhaar, PAN, IFSC, phone, email, DOB, bank account numbers) using regex patterns validated against the Indian financial system format specifications. PII detected at L7 is either redacted or causes a denial depending on the policy configuration for the message type.

9.4 Sovereignty Policy

pkg/sovereignty.ProviderRegistry.Allowed() validates that for a given document classification, the LLM provider being used is permitted under the organisation's data-residency policy. A document classified as 'secret' cannot be sent to a foreign-hosted provider regardless of what the agent requests.

10. Safety Guardrails — Input and Output

10.1 Safety Plugin Architecture

pkg/safety implements a plugin chain where multiple scoring plugins evaluate a message independently and results are aggregated. Two aggregation modes are supported:

- **all-of mode** — all plugins must pass for the message to be allowed. Used for high-stakes operations (payment initiation, KYC verdict).
- **any-of mode** — any plugin passing is sufficient. Used for lower-stakes advisory responses.

The plugin interface is minimal: Score(ctx, message) → (float64, reason, error). Plugins can be LLM-based (sending to an evaluation model) or rule-based (deterministic pattern matching).

10.2 Adversarial Corpus Testing

pkg/safety includes an adversarial corpus — a set of known attack strings covering jailbreak patterns, role-override instructions, and indirect injection payloads. This corpus is run on every CI build to catch regressions in safety scoring. A new model or a new prompt template must pass the full corpus before being promoted.

10.3 Deterministic Agent Cores

The most important safety property is architectural: for high-stakes decisions (KYC verdicts, payment routing, loan approval), the decision logic is deterministic Go code. The LLM produces only the narrative explanation that accompanies the decision — it does not produce the decision itself.

Example — KYC Orchestrator (agents/kyc_orchestrator/):

```
// The risk score is computed deterministically:
score := 0.0
if input.PEPFlag { score += 0.35 } // PEP contribution
if !input.AddressMatch { score += 0.15 } // Address mismatch
// ... other deterministic factors ...

// EDD threshold is a constant, not a model output:
const eddThreshold = 0.70
result.EDDRequired = score >= eddThreshold

// LLM is called only for the explanation:
result.Explanation = llm.Explain(ctx, result) // narrative only
```

11. Data at Rest — Envelope Encryption

Sensitive fields in Postgres are encrypted using AES-256-GCM envelope encryption. Every encrypted value carries a kek_id so individual rows can be re-encrypted on KEK rotation without touching all rows simultaneously.

11.1 Envelope Structure

```
// pkg/crypto/envelope.go
type Envelope struct {
KEKID string // which Key Encryption Key was used
EncDEK []byte // Data Encryption Key, encrypted with KEK
Nonce []byte // AES-GCM nonce for the DEK encryption
Ciphertext []byte // actual plaintext encrypted with DEK
DataNonce []byte // AES-GCM nonce for the ciphertext
}

// Seal encrypts plaintext using a fresh per-row DEK.
func Seal(ctx context.Context, resolver KEKResolver, plaintext []byte)
(*Envelope, error)

// Open decrypts. Looks up KEK by KEKID from resolver.
func Open(ctx context.Context, resolver KEKResolver, env *Envelope)
([]byte, error)
```

11.2 KEK Resolver Interface

The KEKResolver interface is pluggable — any KMS (AWS KMS, GCP Cloud KMS, HashiCorp Vault, or a local test keyring) can be wired in without changing the encryption layer.

```
type KEKResolver interface {
Resolve(ctx context.Context, kekID string) ([]byte, error)
Current(ctx context.Context) (kekID string, key []byte, error)
}
```

11.3 KEK Rotation Playbook

Because every row stores its kek_id, rotation is non-blocking:

- Generate a new KEK and register it in the resolver.
- Set the new KEK as Current — new writes use it immediately.
- Run the rotation job: for each row with the old kek_id, Open then Seal with the new KEK.
- Old KEK can be retired once no rows reference it.

The rotation job is idempotent and safe to re-run after interruption.

12. Tamper-Evident Audit Chain

File: pkg/compliance/audit.go. The audit log is append-only and hash-chained: each entry carries the SHA-256 hash of the previous entry. An insider with DB write access who modifies an audit entry breaks the chain; the verification query detects the break.

```
// pkg/compliance/audit.go
type AuditEntry struct {
ID uuid.UUID
Timestamp time.Time
ActorID string // who performed the action
ActorRole string
TenantID string
Action string // e.g. 'kyc.approve', 'payment.initiate'
ResourceID string
Result string // 'allow' | 'deny'
PolicyRules []string // which rules fired
PrevHash string // SHA-256 of previous entry
Hash string // SHA-256 of this entry (including PrevHash)
}

// Append adds an entry and computes the chain hash.
func (l *AuditLog) Append(ctx context.Context, entry AuditEntry) error

// Verify walks the chain and returns the first break, if any.
func (l *AuditLog) Verify(ctx context.Context) (*ChainBreak, error)
```

12.1 WORM External Sink

For regulatory compliance (RBI Recommendation 22), audit entries are also forwarded to an immutable external sink (Cloud Storage bucket with Object Lock / WORM policy, or Apache Kafka topic with consumer-only ACLs). The internal chain provides fast local verification; the external sink provides the regulator-facing guarantee that the records cannot be deleted.

12.2 Annexure VI — Incident Reporting

File: pkg/incidents/incidents.go. RBI requires regulated entities to file an Annexure VI incident report within specified timeframes for AI-related operational incidents. Genie's incident package models the Annexure VI form as a Go struct and auto-files an incident when:

- An agent returns an error that triggers the fallback path.
- A safety plugin flags a message above the incident threshold.
- A hallucination is detected post-hoc via the verification pipeline.
- A BCP drill fails its success criterion.

```
// pkg/incidents/incidents.go
type AnnexureVI struct {
IncidentID string
DetectedAt time.Time
```

```
ReportedAt time.Time
Category IncidentCategory // hallucination|bias|security|availability
AffectedAgents []string
UserImpact string
RootCause string
Remediation string
FiledBy string
}
```

13. Business Continuity — Fallback Agents and BCP Drills

13.1 Fallback Wiring

Every production agent has a registered fallback. When HandleMessage returns an error, the orchestrator routes a fallback_request message to the fallback agent. The fallback must preserve the original message's tenant metadata.

```
// pkg/orchestration/orchestrator.go
orch := orchestration.NewOrchestrator(reg, bus, policy, env).
SetFallback("kyc_orchestrator", "kyc_orchestrator_fallback").
SetFallback("payment_orchestrator", "payment_orchestrator_fallback")
```

13.2 Forced-Failure BCP Drills

Fallback agents are only as good as the last time they were tested. Genie includes a BCP drill mode that forces specific agents to fail and verifies that the fallback path activates, completes successfully, and files the expected Annexure VI incident report.

```
// Run a BCP drill — forces kyc_orchestrator to fail
// and verifies kyc_orchestrator_fallback handles the traffic
result := bcp.RunDrill(ctx, bcp.DrillConfig{
TargetAgent: "kyc_orchestrator",
FailureMode: bcp.FailTimeout,
DurationSecs: 60,
SuccessCriteria: bcp.Criteria{
FallbackActivated: true,
IncidentFiled: true,
P99LatencyMs: 2000,
},
})
```

14. Observability — Every Decision Is a Span

pkg/observability instruments every policy decision, every agent invocation, every LLM call, and every DB query with an OpenTelemetry span. The span carries the policy decision (allow/deny), the denial reason, the agent ID, the tier, the tenant ID (hashed for privacy), and the latency.

14.1 Key Spans

Span name	Carries
genie.policy.evaluate	CompositePolicy.Evaluate() result for each message
genie.agent.handle	Agent HandleMessage() — includes tier, agent ID, result
genie.db.tenant_query	WithTenant() — includes tenant_id hash, query type
genie.llm.call	LLM API call — includes provider, model, token count, latency
genie.tokenexchange	RFC 8693 exchange — includes actor_id, audience, cache hit/miss
genie.audit.append	Audit chain append — includes chain hash
genie.incident.file	Annexure VI filing — includes incident category

14.2 Security Dashboard SLIs

SLI	What it measures
policy_deny_rate	Denials per minute, broken down by policy type (RBAC/Tenant/Tier/Injection)
cross_tenant_attempt_rate	Messages denied by TenantPolicy — leading indicator of confused-deputy bugs
below_tier_attempt_rate	Messages denied by TierPolicy — non-production agents receiving live traffic
token_exchange_cache_hit_rate	Fraction of exchanges served from cache vs fresh mints
audit_chain_break_alert	Alert fires if Verify() finds a break — immediate P0
llm_sovereignty_violation_rate	Requests denied by sovereignty check — data residency drift

15. RBI FREE-AI Alignment — Recommendation Mapping

RBI's FREE-AI framework comprises 7 Sutras and 26 Recommendations. The table below maps each recommendation to the Genie implementation.

Rec	Title	Status	Genie Implementation
Rec 01	Fairness Assurance	■	pkg/safety plugin chain; adversarial corpus on every release
Rec 02	Risk Classification	■	agent.RiskClass (RiskLow/Medium/High/Critical) + pkg/agent/risk.go
Rec 03	Human Oversight for High-Risk	■	sme_loan_workflow HITL gate; elevation PAM for admin actions
Rec 04	Explainability	■	ExplainabilityPolicy on bus; every LLM call returns explanation token
Rec 05	Auditability	■	Hash-chained audit log; all spans in OTel; Annexure VI auto-filing
Rec 06	Consent Management	■	ConsentPolicy checks user consent before data-processing messages
Rec 07	Data Minimisation	■	PIIPolicy redacts before cross-boundary; sovereignty classification
Rec 08	Accuracy and Reliability	■	Deterministic agent cores; LLM narration-only; verification pipeline
Rec 09	Robustness	■	Fallback agents; BCP drills; circuit breakers; budget wrappers
Rec 10	Security	■	All 11 layers; RLS; RFC 8693; TierPolicy; PromptInjectionPolicy
Rec 11	Privacy by Design	■	Envelope AES-256-GCM at rest; TLS in transit; per-tenant isolation

Rec 12	Bias Detection	■	pkg/safety scoring; full multi-group audit roadmapped
Rec 13	Model Card	■	pkg/web/handlers/aibom.go — AIBOM endpoint; full model card roadmapped
Rec 14	Board-Approved Policy	■	pkg/policy/dsl — CEL-style DSL with board-approved YAML rule files
Rec 15	Data Lifecycle Governance	■	pkg/storage/postgres RLS + 0005_rls.sql + retention.go
Rec 16	Vendor Management	■	pkg/sovereignty.ProviderRegistry; AIBOM provenance tracking
Rec 17	Product Approval (New AI Products)	■	pkg/agent/tier.go — Sketch→Prototype→Beta→Production gate
Rec 18	Incident Response	■	pkg/incidents AnnexureVI struct; auto-filing on agent failure
Rec 19	Customer Grievance Redressal	■	agents/complaint_triage + feedback handler; escalation path roadmapped
Rec 20	Model Risk Management	■	Deterministic cores; LLM narration-only; AgentDojo-style benchmarks
Rec 21	Third-Party AI Risk	■	AIBOM; pkg/safety adversarial corpus on every provider swap
Rec 22	Tamper-Evident Audit Trail	■	Hash-chained audit.go + WORM external sink + RFC 8693 act chain
Rec 23	AI Inventory	■	pkg/web/handlers/inventory.go — /v1/ai-inventory with tier + risk fields
Rec 24	Data Localisation	■	pkg/sovereignty — per-classification provider allowlist
Rec 25	Model Updates	■	Tier promotion gate; BCP drill before production promotion
Rec 26	Stress Testing	■	BCP forced-failure drills; adversarial corpus; load test suite

Legend: ■ Implemented and tested · ■ Partially implemented, roadmapped to full

16. Test Coverage

16.1 Unit Tests — Security Primitives

Package	File	Test cases
pkg/storage/postgres	tenant_test.go	TestEmptyTenantRejected, TestAdminSentinelConstant
pkg/auth	jwt_test.go	TestIssueVerify_Roundtrip, TestVerify_BadSignature, TestVerify_Expired, TestPasswordHashing
pkg/auth/elevation	elevation_test.go	TestRequest_RejectsEmptySubject, TestApprove_SingleApproverActivates, TestApprove_NEyesPending, TestApprove_DuplicateApprover, TestApprove_SelfApprovalRejected (9 tests)
pkg/auth/tokenexchange	exchange_test.go	TestPreservesUserAddsActor, TestRejectsMissingAudience, TestRejectsInvalidSubjectToken, TestRejectsExpiredSubjectToken, TestCachesByTuple, TestInvalidateClearsCacheForUser, TestNestedActorChain (7 tests)
pkg/agent	tier_test.go	TestDeclaredTier, TestDefaultsToPrototype, TestProductionPredicate, TestTierOrdering, TestAtLeastFloor, TestUnknownTierNegativeOrdinal (6 tests)
pkg/governance	tenant_test.go	TestMissingTenant, TestMatchingTenant, TestMismatchedTenant, TestAppliesToFilter, TestAdminBypassOptIn, TestAdminBypassRequired, TestMetaStringSliceCoercion (7 tests)

16.2 Integration Tests — Security Envelope

File: tests/security_envelope_test.go — 8 end-to-end tests wiring TierPolicy + TenantPolicy + RBACPolicy + orchestrator fallback + RFC 8693 token exchange together:

Test	What it proves
TestSecurityEnvelope_SketchTierIsBlocked	TierSketch agent must not handle finance_question — tier denial fires before HandleMessage

TestSecurityEnvelope_MissingTenantIsBlocked	Production-tier agent with no tenant_id is denied by TenantPolicy
TestSecurityEnvelope_CrossTenantIsBlocked	tenant_id ≠ expected_tenant (confused-deputy attempt) is denied
TestSecurityEnvelope_HappyPath	Properly-formed message reaches the production agent with no denials
TestSecurityEnvelope_FallbackTriggers	Agent error → orchestrator routes fallback_request preserving tenant metadata
TestSecurityEnvelope_TokenExchangeAuditIdentity	Two-hop exchange: Subject stays user; Actor chain is kyc_orchestrator → kyc-mcp-server
TestSecurityEnvelope_InvalidatePropagates	Invalidate() clears all cached tokens for the user
TestSecurityEnvelope_FallbackPreservesTenantMetadata	fallback_request carries original tenant_id — no tenant-leak gap in the fallback path

16.3 UI Contract Tests — Security Primitives

File: pkg/web/handlers/ui_security_test.go — 8 tests pinning the security boundary between the UI and the server:

Test	What it proves
TestUI_InventoryFetchGatedByAdmin	/ai-inventory fetch must be inside the isAdmin block; never before it
TestUI_AdminPanelsNotPopulatedInHTML	#inventory, #aibom, #incidents panels must be empty in static HTML
TestInventory_ListIncludesTier	Handler emits 'tier' JSON field; risk team reads this column on dashboard
TestInventory_TierFieldStableJSONName	InventoryItem JSON key is exactly 'tier' — UI column depends on it
TestUI_NoAdminFieldsInPublicHTML	__admin__ sentinel and Postgres RLS internals never appear in HTML/CSS
TestUI_AdminGuardChecksRolesArray	Admin guard uses .includes('admin') on roles array, not role === 'admin'

TestUI_PersistSessionStoresRoles	persistSession() stores state.user (with roles) — admin status survives reload
TestUI_LoginSuccessEntersApp	Both login and signup success paths call persistSession() and enterApp()

16.4 Total Coverage

Category	Count
Unit tests (security packages)	37
Integration tests (security_envelope_test.go)	8
UI contract tests (ui_security_test.go + ui_contract_*.go)	110
Agent-level unit tests (across 60+ agents)	240+
Total Go packages passing go test ./...	104

17. Operational Runbook

17.1 Deploying the RLS Migration

```
# Apply migration (requires BYPASSRLS role for migration user)
psql $DATABASE_URL -f pkg/storage/postgres/migrations/0005_rls.sql

# Verify FORCE is set on all tenant tables
psql $DATABASE_URL -c \
"SELECT relname, relrowsecurity, relforcerowsecurity
FROM pg_class
WHERE relname IN
('documents','accounts','mcp_tokens','incidents','users');"

# Expected output: relrowsecurity=t, relforcerowsecurity=t for all rows
```

17.2 Wiring WithTenant in a New Handler

```
func (h *MyHandler) Get(w http.ResponseWriter, r *http.Request) {
claims := mid.ClaimsFromCtx(r.Context())
err := h.DB.WithTenant(r.Context(), claims.Subject, func(ctx context.Context, tx pgx.Tx)
error {
// All queries in this closure are tenant-scoped automatically
rows, err := tx.Query(ctx, 'SELECT * FROM documents')
// ...
return err
})
// ...
}
```

17.3 Minting a Token-Exchange Service

```
issuer := auth.NewIssuer(secret, 'genie-api', []string{'genie-api'}, time.Hour)
svc := tokenexchange.New(issuer, 'genie-api')

// Call before sending to a downstream MCP server:
exchanged, claims, err := svc.Exchange(ctx, tokenexchange.Request{
SubjectToken: userToken,
ActorID: agentID,
Audience: mcpServerURL,
})

// On user logout or password change:
svc.Invalidate(userSubject)
```

17.4 Promoting an Agent to Production

Checklist — all items must be checked before setting TierProduction:

- Security review completed by at least one non-author engineer.
- Red-team session — attempted prompt injection, tier bypass, cross-tenant request.

- Fallback agent registered and tested via BCP drill.
- Audit hooks verified — every HandleMessage call produces an audit entry.
- Explainability token returned for every high-stakes decision.
- RiskClass set to the appropriate level (not defaulting to RiskLow).
- Agent listed in /v1/ai-inventory with correct tier and risk fields.
- go test ./agents//... passes all 100% of tests.

17.5 Verifying the Security Envelope

```
# Run the full integration test suite:
go test ./tests/... -v -run TestSecurityEnvelope

# Run the UI contract tests:
go test ./pkg/web/handlers/... -v -run TestUI

# Run all 104 packages:
go test ./...

# Expected: all PASS, zero FAIL
```

17.6 Audit Chain Verification

```
// Scheduled daily — alert if non-nil result
break, err := auditLog.Verify(ctx)
if break != nil {
// PAGE IMMEDIATELY — this is a P0 security incident
incident.File(ctx, incidents.AnnexureVI{
Category: incidents.CategorySecurity,
RootCause: 'Audit chain break detected at entry ' + break.EntryID,
})
}
```

Appendix A — Key Files Quick Reference

File	Contents
pkg/auth/jwt.go	HS256 JWT issuance, verification, IssueWithActor()
pkg/auth/types.go	Claims, Actor, Role types; RFC 8693 act/nested actor chain
pkg/auth/tokenexchange/exchange.go	RFC 8693 token exchange service, cache, invalidation
pkg/auth/elevation/elevation.go	Two-person-rule privileged access manager
pkg/auth/device_flow.go	OAuth 2.0 Device Authorization Grant (RFC 8628)
pkg/auth/webauthn.go	WebAuthn passkeys with Ed25519 authenticators
pkg/agent/tier.go	TierSketch/Prototype/Beta/Production; TierOf(); AtLeast()
pkg/agent/risk.go	RiskLow/Medium/High/Critical; risk class per-agent
pkg/governance/tenant.go	TenantPolicy — bus-layer tenant isolation
pkg/governance/rbac.go	RBACPolicy — role-based access on message types
pkg/governance/compliance.go	CompositePolicy — chains all governance checks
pkg/governance/prompt_injection.go	PromptInjectionPolicy — deterministic injection detection
pkg/governance/pii.go	PIIPolicy — Aadhaar/PAN/IFSC/phone pattern detection
pkg/governance/sovereignty.go	SovereigntyPolicy — provider/classification allowlist
pkg/storage/postgres/tenant.go	WithTenant(), WithAdminContext(), ErrNoTenant
pkg/storage/postgres/migrations/0005_rls.sql	FORCE ROW LEVEL SECURITY on all tenant tables
pkg/compliance/audit.go	Hash-chained tamper-evident audit log
pkg/incidents/incidents.go	AnnexureVI struct; auto-filing on agent failure
pkg/identity/identity.go	SPIFFE DID + W3C Verifiable Credential (Ed25519)
pkg/policy/dsl/	Board-approved policy DSL (CEL-style, stdlib-only)
pkg/safety/	Plugin chain; adversarial corpus; all-of / any-of aggregation
pkg/sovereignty/	ProviderRegistry; per-classification data-residency rules

pkg/llm/	Deadline / circuit / budget / router wrappers
tests/security_envelope_test.go	8 integration tests wiring all 4 Q1 primitives
pkg/web/handlers/ui_security_test.go	8 UI contract tests for security boundary
pkg/web/handlers/inventory.go	InventoryItem with Tier field for risk dashboard
pkg/web/mid/auth.go	RequireAuth, RequireRole HTTP middleware

Appendix B — Glossary

Term	Definition
AdminTenant	The __admin__ sentinel value for app.current_tenant. Unlocks cross-tenant reads in RLS policies. Used only at login and admin-only endpoints.
Actor (RFC 8693)	The agent or service currently acting on a user's behalf. Carried in the 'act' JWT claim. Nested actors form the full audit chain.
CompositePolicy	The ordered chain of all governance policies evaluated on every bus message. First denial wins; the message is dropped and the denial is recorded.
DEK	Data Encryption Key — per-row symmetric key, itself encrypted with the KEK.
Dual-identity token	A JWT where Subject = the human user and Actor = the agent, per RFC 8693. Downstream services can audit both identities from a single token.
GUC	Grand Unified Configuration — Postgres session parameter. app.current_tenant is set via SET LOCAL (transaction-scoped).
KEK	Key Encryption Key — the master key held in the KMS. Encrypts DEKs; never encrypts data directly.
looseVerifier	A wrapper around the JWT Issuer that skips audience validation. Used by token-exchange for multi-hop chains where the audience changes per hop.
RLS	Row-Level Security — Postgres feature that filters rows based on session GUC values. FORCE extends the policy to table owners.
SPIFFE/SVID	Secure Production Identity Framework for Everyone / SPIFFE Verifiable Identity Document. Workload identity standard; Genie uses a compatible DID+VC toolkit.

TierPolicy	A policy that denies dispatch to any agent below TierProduction. Wired into the bus CompositePolicy.
Toxic agent flow	A multi-message sequence where an early injected instruction causes harmful behaviour in a later step. Detected by sequence-aware policies.
WORM	Write Once Read Many — immutable storage used for the external audit sink.